

Objective

The goal of this exercise is to design and implement a C++ framework for basic vector operations. The framework must rely on inheritance, polymorphism, and templates, and expose its functionality to Python using pybind11.

Background

Let $x, y, z \in \mathbb{R}^n$ be vectors and $\alpha \in \mathbb{R}$ a scalar. The following operations are defined:

- **Vector addition:** $z_i = x_i + y_i$.
- **Scalar multiplication:** $y_i = \alpha x_i$.
- **Dot product:** $x \cdot y = \sum_i x_i y_i$.
- **Euclidean norm:** $\|x\|_2 = \sqrt{\sum_i x_i^2}$.

All operations are element-wise except for the dot product and the norm.

Instructions

You are asked to build a reusable vector operations library in C++. The library must allow different operations to be applied to vectors through a common interface and must support different numeric types using templates.

Tasks

1. (2 points) **Define a templated Vector class**

Define a templated class `Vector<T>` that:

- Stores a dynamic array of elements of type `T`.
- Provides constructors for specifying the vector size.
- Provides a method `size()` to query the dimension of the vector.

2. (2 points) **Element access and validation**

- Implement element access using `operator[]`.
- Ensure const-correctness.
- Accessing invalid indices must throw an exception.
- Follow RAII principles and ensure memory safety.

3. (3 points) **Vector operation hierarchy**

- Define an abstract base class:

```
template<typename T>
class VectorOp {
public:
    virtual Vector<T> apply(const Vector<T>& v) const = 0;
    // ...
};
```

- Implement the following derived classes:

1. `ScaleOp<T>`: Performs multiplication by a scalar (provided at construction).
2. `AddOp<T>`: Adds a second vector (provided at construction).
3. `NormalizeOp<T>`: Returns a normalized vector (i.e., divided by its Euclidean norm).

- Each derived class must:
 - Override `apply()`.

- Validate dimensions where required.
- Throw exceptions on invalid operations (e.g., size mismatch or zero norm).

4. (2 points) Dot product operation

Implement a `DotOp<T>` class that computes the dot product between two vectors.

- Clearly define how the scalar result is returned.
- Validate that the vectors have the same dimension.
- Handle invalid cases with exceptions.

5. (1 point) Testing and polymorphism

In `main()`:

- Demonstrate the use of polymorphism by applying operations through a base-class pointer or reference.
- Verify that the framework works for at least two different numeric types (e.g., `int`, `double`).

6. (1 point) Configuration and compilation

- Write a `CMakeLists.txt` file to compile your code into a reusable library and a test executable.
- Include proper C++17 standard requirements and optimization flags.
- Provide clear build and usage instructions.

7. (4 points) Python bindings using `pybind11`

- Expose the `Vector` class and all operations to Python.
- Ensure that C++ exceptions are properly translated into Python exceptions.
- Write a Python script that:
 - Creates vectors.
 - Applies vector operations.
 - Computes dot products.
 - Compares the results and performance with NumPy on large vectors.
 - Briefly comments on any observed differences.

Evaluation criteria

- Correct use of templates, inheritance, and polymorphism.
- Correctness of vector operations.
- Proper handling of edge cases and exceptions.
- Memory safety and RAII compliance.
- Clean and usable Python bindings.
- Meaningful comparison with NumPy.
- Quality and clarity of the CMake configuration.
- Code organization and readability.